WHEN DO THE NEW #SAT ALGORITHMS BEAT BRUTE FORCE? CROSSOVER POINTS OF THE PROVEN BOUNDS BEHIND THE ETH REFUTATIONS

VAIBHAV TULSYAN

Abstract. In October 2026 OpenAI and Anthropic claimed deterministic algorithms that count the satisfying assignments of a k-CNF in time $2^{O(n \log\log n/\sqrt{\log n})}$ or similar, refuting the Exponential Time Hypothesis. However, neither manuscript computes from which n on its *proven* bound is smaller than brute force. We compute this crossover n^* for the bounds as they are proved, and report $t^* = \log_2 n^*$. We keep every explicit term and constant, treat existential constants as parameters, and evaluate each bound exactly in the logarithmic domain. For Anthropic's official Lean main theorem on 3-CNFs with $m = 4.26n$ clauses, $t^* = 552\,040$ if the unknown polynomial degree q of its Turing-machine time bound is 1, and t^* grows roughly like q^2; re-optimising the free parameters of the same lemmas gives 245 009. For OpenAI's interpolation algorithm, $t^* \approx 3.02 \cdot 10^{13}$ as written and $\approx 8.53 \cdot 10^7$ with re-optimised parameters, mostly by using an exact binomial tail in place of a Hoeffding window. Our exact algorithm for counting Hamiltonian cycles, which rests on Anthropic's manuscript, beats Held–Karp on cubic graphs from $t^* = 55\,440$. Anthropic's 3-SAT decision machine searches exhaustively for every $n < 2^{2^{1722}}$. In every Anthropic-based case n^* is a staircase, fixed by the first recursion level at which a single clique exponent falls below n. These numbers are computations about proven upper bounds, not theorems and not running times; none is machine-checked. They are conditional on the claimed refutations, which are under review, and nothing practical follows. The script that computes them is in our public repository.

1. Introduction

The Exponential Time Hypothesis (ETH) of Impagliazzo and Paturi [5] asserts that 3-SAT on n variables needs time $2^{\delta n}$ for some $\delta > 0$. In October 2026 two manuscripts, by OpenAI [7] and by Anthropic [1], each claimed a deterministic algorithm that counts the models of a k-CNF in subexponential time. Both come with Lean 4 formalisations of the top-level statements. We write [A] and [O] for them. The Main Theorem of [A] gives time $2^{O(\min\{km/\sqrt{\log m},\, k^2 n \log\log n/\sqrt{\log n}\})}$. [O] gives $(n+m)^{O_k(1)} 2^{O_k(n \log\log n/\sqrt{\log n})}$ bit operations, and $2^{O_{k,D}(n/\sqrt{\log n})}$ when $m \le Dn$. These are *claimed* results, under review. In a companion paper [8] we derived from [A] an exact deterministic algorithm that counts the Hamiltonian cycles (#HC) of a graph with n vertices and $m = O(n)$ edges in time $2^{O(n/\sqrt{\log n})}$. The Lean development and the scripts used here are in our public repository, `https://github.com/xennygrimmato/eth-hamiltonicity` [9].

However, an $O(\cdot)$ in an exponent says nothing about when the algorithm first wins. The question is not idle. The sources fix almost all of their constants (the Lean development of [A] fixes every schedule parameter), so the bound they prove is a concrete function of n, up to a few existential constants. Neither manuscript computes where its concrete bound crosses brute force (the Lean development of [A] records only that its decision machine searches exhaustively below $2^{2^{518}}$, see Section 4). We are not aware of such a computation, and we do it here.

1.1. The crossover problem.

We reserve the word *crossover* for a proven bound, never for a running time.

Definition 1.1. Let $T_{\mathrm{alg}}(n)$ be a proven upper bound on the number of steps of an algorithm on inputs with n variables (or vertices), with every explicit term and polynomial factor kept and the free parameters chosen as the tier says (Section 2.3), and let $T_{\mathrm{bf}}(n)$ be the brute-force bound: 2^n for #3-SAT and $n^2 2^n$ (Bellman,

Date: November 2026.

2020 *Mathematics Subject Classification.* 68Q25, 68W40, 68Q17.

Key words and phrases. Exponential Time Hypothesis, model counting, fine-grained complexity, explicit constants, galactic algorithms.

Held–Karp [2, 3]) for #HC. The *crossover* n^* is the least n with $T_{\mathrm{alg}}(n) < T_{\mathrm{bf}}(n)$. Since n^* has between 10^3 and 10^{14} binary digits (and 2^{1747} for one machine), we report $t^* = \log_2 n^*$.

We write $\rho_{\mathrm{alg}}(n) = \log_2 T_{\mathrm{alg}}(n)/n$ for the *rate*. For #3-SAT the crossover is where the rate first drops below 1.

Example 1.2 (Running example)**.** Take the official Lean main theorem of [A] on 3-CNFs at the satisfiability threshold density $m = 4.26n$, with $q = 1$. Its exponent is dominated by a single term, the clique exponent e'_ℓ of the recursion level ℓ times the network total $U \approx 2km$. So the rate is about $2e'_\ell k\alpha = 25.56\, e'_\ell$ with $\alpha = 4.26$. The rate first drops below 1 at level $\ell = 10$, where $e'_{10} = 1/32$ gives $25.56/32 = 0.79875$. Level 10 may be used only once $K_{10}^9 \le m$. Hence $t^* = 9 \log_2 K_{10} - \log_2 4.26 = 552\,042 - 2.09$, that is $t^* = 552\,040$. We return to this example in Section 3.1.

1.2. **The naive route and the principle.** At first one might read the crossover off the stated forms. However, these hide every constant in an $O(\cdot)$, so they cannot fix a crossover. Instead, we evaluate the bound *as proved*: we transcribe it from the Lean definitions and the TEX sources, keep every explicit term, and evaluate it exactly in the base-2 logarithmic domain (Section 2.1). Each existential constant stays a parameter. We report a table in it, or the break-even value that it would have to exceed to move n^*.

The key observation is that, for every bound built on [A], the crossover is a staircase. Example 1.2 shows the mechanism. The rate is the clique exponent times a network total, to high accuracy, and the clique exponent halves every two levels, while $\log_2 K_\ell$ doubles with every level. So a change of constant helps only once it crosses a level boundary, and then it halves t^*.

Table 1 is the summary.

TABLE 1. Crossover $t^* = \log_2 n^*$ of the proven bounds. A: as written; B: same lemmas, free parameters re-optimised; C: constant changes justified outside the formal proofs; [H]: heuristic. Rows 1–3 use $q = 1$ for the unknown degree of the Turing-machine time bound (the explicit-part lower bound, Section 2.5); for other q see Table 6. In rows 7–8 the values in parentheses use the degree network of [9].

	algorithm (brute force)	A	B	C
1	[A], official Lean thm., #3-SAT, $m = 4.26n$ (2^n)	552 040	245 009	119 103; 31 358 [H]
2	[A], n-route, #3-SAT, any m (2^n)	$3.42 \cdot 10^{10}$	$1.61 \cdot 10^{10}$	$3.93 \cdot 10^9$; $1.05 \cdot 10^9$ [H]
3	[A], 3-SAT decision machine (2^n)	2^{1747}	—	—
4	[O], Engine I, #3-SAT, any m (2^n)	$3.02 \cdot 10^{13}$	$8.53 \cdot 10^7$	$3.82 \cdot 10^7$
5	[O], Engine II (2^n)	only a lower bound: $\log_2(\text{wires}) \ge 2^{85}$ (2^{510} at full depth)		
6	#HC, cubic graphs ($n^2 2^n$)	55 440	44 356	20 888; 4 905 [H]
7	#HC, $\Delta = 4$ ($n^2 2^n$)	112 860	90 291	43 205; 10 470 (9 799) [H]
8	#HC, $\Delta = 8$ ($n^2 2^n$)	458 100 (112 860)	366 482 (90 292)	177 700 (49 161); 46 182 (19 350) [H]

So, for example, the official Lean theorem of [A] first certifies a win over 2^n on 3-CNFs at the satisfiability threshold density at about $n = 2^{552\,040}$, and only if $q = 1$. With $q = 2$ it is $2^{2.21 \cdot 10^6}$, and with $q = 10$ it is $2^{3.54 \cdot 10^7}$. OpenAI's Engine I as written needs $n \approx 2^{3.02 \cdot 10^{13}}$. Our #HC algorithm, which has no Turing-machine degree because it counts word operations, has the smallest crossover of all.

1.3. **Contributions.** Our contributions are:

- We define the crossover of a proven bound and three tiers of it: the bound as written, the same lemmas with re-optimised free parameters, and constant changes justified outside the formal proofs (Section 2).
- We compute the crossovers of the official Lean main theorem of [A], of its route for any number of clauses, and of its 3-SAT decision machine, and we show how they depend on the degree q (Sections 3.1 to 3.3).
- We compute the crossovers of our #HC algorithm against Held–Karp, on graphs of maximum degree 3, 4 and 8 (Section 3.4).
- We compute the crossover of Engine I of [O], as written and re-optimised, and find that Engine II as written yields only a lower bound (Sections 3.5 and 3.6).

- We rank the levers that move the crossovers (Section 3.7), and record four facts about the sources, each of which changes a number (Section 4).
- We describe the script that computes every number, and its reproduction (Section 7).

This is a small step. We make no claim of practicality, and Section 6 lists what the numbers do not say.

1.4. **Status of each claim.** This note reports computations, not theorems. We separate four kinds of claim. *Computed numerically*: every crossover t^*, every rate and every term in the tables and in the text. Each is the output of an exact computation (Sections 2.1 and 7) applied to formulas that we transcribed from the sources (file and lemma references below). The transcription is ours, the computations are not formally verified, and none of the numbers is a theorem, in Lean or on paper. *Proved on paper*: only short arguments about the formulas, namely that the explicit-part values are lower bounds on the crossovers of the proven bounds (Section 2.5), and the derivation of the Engine II threshold from its parameter regime (Section 3.6). *Machine-checked*: nothing in this note. The Lean theorems of [A] and of [8] state the bounds that we evaluate, but no Lean proof checks our transcription or our numbers. *Assumed*: the claimed results of [A] and [O], which are under review; for #HC, the three internal results of [A] on which [8] rests, and the explicit bounds of our research notes; and, in the entries marked [H], changes of constants that nobody has proved. Where a number depends on a constant that the sources only assert to exist, we say so and report its sensitivity. A crossover of a proven bound says where the *guarantee* first beats brute force. The algorithms might do better, and the guarantees might be improvable, which is what tiers B and C measure.

Outline. Section 2 describes the method, Section 3 the results, Section 4 four facts about the sources, Section 5 related work, Section 6 the limitations, and Section 7 the reproduction.

2. Method: exact evaluation of the proven bounds

In this section we describe how we evaluate a bound, give the schedule of [A] on which most bounds rest, define the tiers, and list the existential constants and how we handle them.

2.1. **Exact evaluation in the log domain.** We represent all quantities by their base-2 logarithms and evaluate them with mpmath [6] at 60 significant digits. The level sizes $K_\ell = 2^{L_\ell}$ have L_ℓ up to about 10^9 and are never built as integers. The exponents L_ℓ, the level rules and the Lean functions `Nat.size`, `Nat.clog`, floors and ceilings are evaluated exactly on these representations. We put $n = 2^t$ with real t, and $m = \alpha n$ for the m-route. Within one band of t on which the recursion level is fixed the rate decreases in t, and we find the crossover by bisection in t to relative precision 10^{-6}. Tier B and C crossovers are minimised over all levels. Terms reported as "below $10^{-x} \cdot n$" are printed by the script in decimal notation.

2.2. **The schedule of [A].** The algorithm of [A] is a recursion over levels ℓ. Its Lean development fixes the schedule (`Levels.Gen`, `statements/LevelsStatement.lean`; all Lean paths are relative to the `proof/` directory of [A]):

$$K_0 = 2^{x_0}, \quad g_\ell = 2^{3\ell+x_0} K_\ell, \quad K_{\ell+1} = g_\ell^2, \quad \zeta_\ell = 2^{-\ell-5}, \quad e_0 = \tfrac{1}{2}, \quad e_{\ell+1} = (\rho + 3\zeta_\ell)\, e_\ell + \frac{3}{K_\ell}, \tag{2.1}$$

with $\rho = 1/\sqrt{2}$. The official route uses $x_0 = 18$, the least value allowed (`maintheorem/CountedRoute/Defs.lean`). Then $\log_2 K_\ell = 60 \cdot 2^\ell - 6\ell - 42$, which the script asserts for $\ell < 40$ together with condition (ii) of the grouping lemma, $K_\ell \ln(2 g_\ell K_\ell) \le 2\zeta_\ell^2 g_\ell$. The number e_ℓ is the clique exponent: the clique algorithm at level ℓ runs in $2^{e_\ell K^2 \lambda}(K(\lambda + B))^{C_0 K^2}$ steps (`statements/CliqueLevelsStatement.lean`). The official main theorem, however, does not use e_ℓ but the rational number e'_ℓ returned by `num`/`den` in `rates/GeneralPieces/Num/Between.lean` (lines 34–37): $e'_\ell = 2^{-\ell/2}$ for even ℓ and $1.4 \cdot 2^{-(\ell+1)/2}$ for odd ℓ (Section 4).

The ratio e_ℓ/ρ^ℓ tends to $0.644772 = \frac{1}{2}\prod_j (1 + 3\zeta_j/\rho)$. The terms $3\zeta_\ell$ cost a factor 1.29 over $\rho^\ell/2$, while the terms $3/K_\ell$ are below 2^{-17} at $\ell = 0$ and doubly exponentially smaller afterwards. Two facts drive everything below: e_ℓ halves every two levels, and $\log_2 K_\ell$ doubles with every level.

TABLE 2. The Lean schedule at $x_0 = 18$ (the last column is the ratio of the printed values).

ℓ	$\log_2 K_\ell$	e_ℓ	e'_ℓ	e_ℓ/ρ^ℓ	e'_ℓ/e_ℓ
0	18	0.5	1.0	0.5	2.0
2	186	0.3019	0.5	0.6038	1.66
4	894	0.1586	0.25	0.6342	1.58
6	3 762	0.08026	0.125	0.6421	1.56
8	15 270	0.04026	0.0625	0.6441	1.55
9	30 624	0.02848	0.04375	0.6444	1.54
10	61 338	0.02014	0.03125	0.6446	1.55
12	245 646	0.01007	0.01563	0.6447	1.55

2.3. **Tiers and labels.** *Tier A* is the bound as written: the Lean definitions for [A], and the paper's parameter choices for [O] and for [8]. *Tier B* uses the same proven lemmas with their free parameters re-optimised for each n. For [A] these are the level and the numerator e_ℓ in place of e'_ℓ, both allowed by other Lean routes of [A]. For [O] they are the parameters s, t, r, Q, R, τ, subject only to the numerical hypotheses of `lem:layout`. *Tier C* changes constants in ways justified outside the formal proofs. C1 re-tunes $\zeta_\ell = 2^{-(\ell+z)}$ and takes the least g_ℓ/K_ℓ that condition (ii) allows; this rests on the written proof of [A], not on Lean. For [O], tier C replaces the bound $m_{\text{test}} \le 6n$ by $m_{\text{test}} \le n + n/\log_2(4/3) + 2 \approx 3.41n$, which the proofs of `lem:sparse-checks` and `lem:greedy-hashes` give but the paper does not state. Items marked [H] are heuristic. C2 and C3 assume a base case of the clique recursion with exponent $e_0 = 5/18$ (= 2.5/9) or $e_0 = \omega/9 = 0.2635$ with $\omega = 2.371552$ [10], as a fast-matrix-multiplication base case might give. C4 adds $x_0 = 10$ (which needs the pieces constant $c \le 1017$). C5 drops the K^g grouping search. None of the [H] changes is proved, in the sources or here.

2.4. **Explicit and existential constants.** Table 3 lists every constant that is existential in Lean or hidden in an $O(\cdot)$ of the sources, and how we handle it. We never give an existential constant a value. Each is a parameter, and we report either a table in it or the *break-even* value it would have to exceed to move n^*.

TABLE 3. Existential and hidden constants.

constant	where	effect on n^*
q, degree of the TM time bound [A]	`Computes` (`variables/PairsToBound/Defs.lean`, l. 192): steps $\le q(\lvert x\rvert + 2^{\text{expo}} + 1)^q$; folded into the C of the final forms	**the only one that matters**: crossover needs rate $< 1/q$; t^* grows about 4× per doubling of q (Table 6)
C_0, clique factor [A]	$\forall c\, \exists C_0$ in `ExplicitCost.RecursionSolves`; enters `needOf` as $C_0 K^2$ size$(\cdot)$	none: tables for $C_0 \in \{0, 1, 10\}$ coincide; at the tier-A n^* of row 1 it matters only if $C_0 > 2^{429\,342}$
C_1, grouping factor (Lemma 8 of [A])	$\exists C_1$ in `GroupingExplicit`; absent from the Lean pad exponent	none for [A]; for #HC (cubic, tier A) only if $C_1 > 2^{55\,421.7}$; C_0 there only if $C_0 > 2^{51\,725.6}$
C_2, the $O(\log n)$ in $\log \tau$ (#HC)	[9], `research/exact-hc-theorem.md` (6.1)–(6.2)	none: only if $C_2 > 2^{55\,421.0}$
$n_0 \le n_1$, "large n" of the n-route [A]	`ThmVariablesAForEveryMachine`: $\exists n_0\, \forall n_1 \ge n_0$	cannot be bounded; can only increase n^*, so the n-route values are lower bounds
$(n+m)^{O_k(1)}$ and other polynomial factors [O]	throughout	none: a factor $(n+m)^c$ matters only if c is of order 2^{λ^*}, $\lambda^* \approx 3.0 \cdot 10^{13}$
Engine II constants $O_C(\cdot)$, $O_{C,A}(\cdot)$ [O]	`thm:tensor-contraction`	not boundable; only an explicit parameter threshold is reported

2.5. **Explicit-part lower bounds.** Setting $C_0 = C_1 = C_2 = 0$ and $q = 1$ and ignoring n_0, n_1 only decreases the bounds. So the resulting t^* is a rigorous lower bound on the crossover of the proven bound. We also drop

$\log_2 q$ and the input length $|x|$ in $q(|x| + 2^{\text{expo}} + 1)^q \geq 2^{q\cdot\text{expo}}$, which again errs downwards. The padded Lean machines of [A] write a pad of 2^{expo} units and fall back to exhaustive search when expo $> n$ (`planM`, `plan`). For them the value is also a lower bound on where the machine itself leaves exhaustive search. For [O] Engine I and for #HC the remaining constants are so far from their break-even values that the lower bound *is* the crossover of the proven bound. For the SAT routes of [A] it is the value at $q = 1$, and the true value is the q-dependent one of Tables 6 and 7.

3. Results: crossover points

In this section we take the bounds one at a time: the three routes of [A], our #HC algorithm, and the two engines of [O]. Each subsection names the dominant term at the crossover. We end with the levers that move the crossovers.

3.1. [A], the official main theorem (m-route): one term fixes the crossover.

`Solution.lean` (line 44) defines `MainResults.mainTheorem` as `CountedRoute.mainTheoremHolds`. Its exponent in m is `needTop` (`rates/ClauseBound/Alg/Top.lean`). Let $K = K_\ell$ for the largest ℓ with $K_\ell^9 \leq m$ (`root9`), $g = 3K^8$, $P = \max(1, 2k\lceil m/g\rceil)$ and $\lambda = \lfloor (K^3 + 2kmK + gP)/K^3 \rfloor$. Then

$$\text{needTop} = \max\Big(3K^8 \operatorname{size} K,\ P\big(\tfrac{2g}{K} + 1\big),\ (K-1)\lambda,\ \lceil e'_\ell K^2 \lambda\rceil + C_0 K^2 \operatorname{size}\big(K(\lambda + B + 1)\big) + 2\Big), \tag{3.1}$$

the last term being `needOf` of `rates/CountedToPadded/Defs.lean` (line 148) with $B = 2gP/K$. The machine runs in at most $q(|x| + 2^{\text{needTop}} + 1)^q$ steps, and searches exhaustively when needTop $> n$. Table 4 shows the rate at $q = 1$, Table 5 the crossovers, and Table 6 the dependence on q.

Table 4. Rate needTop/n of the official theorem, tier A, $m = 4.26n$, $q = 1$, $C_0 = 0$, at $n = 2^{2^j}$. From $j = 8$ on, the solver term `needOf` is the maximum in (3.1).

j	8	12	16	18	20	24	30
level ℓ	0	3	6	8	10	14	20
rate	25.56	8.946	3.195	1.597	0.7987	0.1997	0.02496

Table 5. Crossover t^* of the m-route, $k = 3$, $m = \alpha n$, $q = 1$; ℓ^* is the level at the crossover for $\alpha = 4.26$. C1 is justified by the written proof of [A], not by Lean.

tier	$\alpha = 1$	$\alpha = 3$	$\alpha = 4.26$	ℓ^*
A: `CountedRoute`, `root9`, e'_ℓ ($C_0 \in \{0, 1, 10\}$)	33 858	275 614	552 040	10
B: same lemmas, e_ℓ, level free	7 163	122 176	245 009	9
C1: ζ_ℓ and g_ℓ/K_ℓ re-tuned	6 459	59 247	119 103	8
C2: C1 + $e_0 = 5/18$ [H]	1 449	15 405	31 358	6
C3: C1 + $e_0 = \omega/9$ [H]	1 353	14 412	29 341	6
C4: C2 + $x_0 = 10$ [H]	1 161	13 036	26 621	6
C5: C2 without the K^g grouping search [H]	1 265	13 473	27 431	6

Table 6. Sensitivity of t^* to the Turing-machine degree q, m-route, $\alpha = 4.26$.

tier	$q = 1$	$q = 2$	$q = 3$	$q = 5$	$q = 10$
A	$5.52 \cdot 10^5$	$2.21 \cdot 10^6$	$4.42 \cdot 10^6$	$8.85 \cdot 10^6$	$3.54 \cdot 10^7$
B	$2.45 \cdot 10^5$	$9.82 \cdot 10^5$	$1.97 \cdot 10^6$	$3.93 \cdot 10^6$	$1.57 \cdot 10^7$
C1	$1.19 \cdot 10^5$	$4.78 \cdot 10^5$	$9.58 \cdot 10^5$	$3.55 \cdot 10^6$	$1.42 \cdot 10^7$
C2 [H]	$3.14 \cdot 10^4$	$1.27 \cdot 10^5$	$2.72 \cdot 10^5$	$9.58 \cdot 10^5$	$3.83 \cdot 10^6$

Dominant term. At every tier the cost at n^* is the solver term $\lceil e'K^2\lambda\rceil \approx 2e'_\ell km$, that is, the clique exponent times the network total $U \approx 2km$. We return to the running example (Example 1.2). At the tier-A crossover ($\alpha = 4.26$, $\ell = 10$) the solver term equals $0.79875\,n$, while the grouping search, $P(2g/K+1)$ and $(K-1)\lambda$ are each below $10^{-18459}\cdot n$. So the rate is $2e'_\ell k\alpha$ to that accuracy, and two discrete facts fix n^*. One is the first level with $q\cdot 2e'_\ell k\alpha < 1$ (here $e'_{10} = 1/32$, giving $25.56/32 = 0.79875$). The other is the least n at which that level may be used. In tier A this is the rule $K_\ell^9 \le m$, so $t^* = 9\log_2 K_{10} - \log_2 4.26 = 552\,042 - 2.09$, the value in Table 5. In tier B the binding constraint is the search term $3K^8$ size $K \le n$, so $t^* \approx 8\log_2 K_9$. Since $\log_2 K_\ell$ doubles per level and e_ℓ halves every two levels, n^* is a staircase: a change of constant helps only once it crosses a level boundary, and then it halves t^*. Each doubling of q costs two levels, hence the factor 4 in Table 6 (the steps at $q = 3$ and $q = 5$ are the intermediate odd levels).

3.2. **[A], the n-route (any number of clauses): the same term, at a later level rule.** For formulas with arbitrarily many clauses, [A] first sparsifies and then runs a network whose total is $U \le 6k(n + L_\Delta) + 28kn\lceil\log_2(96\Delta L_\Delta)\rceil + 2(3K^8)^2$, with $L_\Delta = \lceil\log_2 2\Delta\rceil$ and $\Delta = \min(\texttt{occBound}, n+1)$ (`variables/VariableBound/Net/Defs.lean`, `variables/RecursionToBound/Defs.lean`). Here $\texttt{occBound} = 2(\theta_0+\theta_1+\theta_2)+2 = 32\alpha^2+4\alpha+6$ with $\alpha = \lceil 8\varepsilon^{-1}\lceil\log_2(128/\varepsilon)\rceil\rceil$ (`sparsification/SparsificationLemma/Cal/Defs.lean`) and the sparsification parameter $\varepsilon = e'_\ell$ (`epsOf`). The exponent is the solver exponent for this network plus a walk term $\lceil\varepsilon n\rceil + 2 +$ size(stack), at the level given by $K_\ell^{17} \le n$ (`root17`).

TABLE 7. n-route: rate at $n = 2^{2^j}$ (tier A, $q = 1$), and crossovers t^*.

j	22	28	34	37	40
level ℓ	12	18	24	27	30
rate	55.42	8.076	1.153	0.4251	0.1621
U/n	3 546	4 134	4 722	4 974	5 310
tier	$q=1$	$q=2$	$q=5$	$q=10$	
A (`root17`, $\varepsilon = e'_\ell$), $\ell^* = 25$ at $q=1$	$3.42\cdot10^{10}$	$1.37\cdot10^{11}$	$1.10\cdot10^{12}$	$4.38\cdot10^{12}$	
B (e_ℓ, level free)	$1.61\cdot10^{10}$	$6.44\cdot10^{10}$	$5.15\cdot10^{11}$	$2.06\cdot10^{12}$	
C1 (re-tuned, ε decoupled from e)	$3.93\cdot10^{9}$	$1.68\cdot10^{10}$	$1.26\cdot10^{11}$	$5.37\cdot10^{11}$	
C2 (C1 + $e_0 = 5/18$) [H]	$1.05\cdot10^{9}$	$7.28\cdot10^{9}$	$3.36\cdot10^{10}$	$2.33\cdot10^{11}$	

At the tier-A crossover ($q = 1$, $\ell = 25$) the cap is $\Delta = 2.8049\cdot10^{13}$ and $U/n = 4806$. The solver term is $0.82134\,n$, the walk term $1.709\cdot10^{-4}\,n$, and every K-dependent term is below $10^{-6\cdot10^8}\cdot n$. The dominant term is again $e'_\ell U$, and $t^* = 17\log_2 K_{25}$. Lean ties the sparsification parameter to the clique exponent. Decoupling them (tier C1, best at $\varepsilon \approx 128e_\ell$) is worth about two levels, because the walk term εn is cheap while U depends on ε through $\log_2\Delta \approx 2\log_2(1/\varepsilon)$. The existential threshold n_1 can only increase these values.

3.3. **[A], the 3-SAT decision machine: the padding constant dominates.** The rate machine `Sat3.inRateTM` (`rates/PaddedDescent/`) chooses $s = \lfloor 3(\lfloor\log_2\lfloor\log_2 n\rfloor\rfloor - 911)/19\rfloor$ (`sOf`, `Params.lean` line 56). It searches exhaustively while $s < \texttt{sOK}\,3 = 128$, that is, for every $n < 2^{2^{1722}}$. Beyond that it writes a pad of $2^{2^{131}(\lfloor n/2^s\rfloor+1)}$ units ($\texttt{COK}\,3 = 8\cdot2^{128} = 2^{131}$), and its time $q\cdot(\text{pad})^q$ is below 2^n only once $q\cdot2^{131-s} < 1$:

q	1	2	10	1000
needs $s \ge$	132	133	136	142
n^*	$2^{2^{1747}}$	$2^{2^{1754}}$	$2^{2^{1773}}$	$2^{2^{1811}}$

This is a decision procedure, not a counter. Because of its padding constant it is far worse than the counting routes.

3.4. **Our #HC algorithm: no Turing-machine degree.** The #HC algorithm of [8] is conditional on [A] (Theorem A(b), Lemma 8 and the existence of its pieces). It counts word-RAM operations, so there is no Turing-machine degree and no padding. We use the explicit bounds recorded in the repository [9] (`research/exact-hc-theorem.md`, (6.1)–(6.2)): $gP \le 2.33(2m+n) + 3g^3$, $U \le 4.65m + g^3$, $B \le 2(2m+$

$n)/g+5$, $\log\tau \le 1.5(2m+n)/g+3+C_2\log n$, at most $n^{4m/\Theta}$ branches with $\Theta=\max(2,\lceil n/g\rceil)$, and the level rule $K_\ell^{30}\le n$. The log of the bound is

$$1+\max\bigl\{g\log K,\ e_\ell(U+gP/K+K^2)+C_0K^2\log(K(L+B'))+2gP/K+\log\tau\bigr\} \\ +C_1\log(g(P+B))+C_2\log n+\log\#\text{branches},$$

which we compare with $\log_2(n^2 2^n)$. The degree network (`research/hc-degree-theorem.md`) has $U\le 2n\log_2(2e\bar d)+g^3$ and no branching. For Δ-regular graphs we take $\bar d=\Delta$.

TABLE 8. #HC versus $n^2 2^n$: crossover t^*, $m=\Delta n/2$. Tier A uses $K_\ell^{30}\le n$, the Lean schedule and e_ℓ; $C_0=C_1=C_2\in\{0,10\}$ give identical values. The network of [8] has $U\approx 4.65m$, the degree network $U\approx 2n\log_2(2e\Delta)$.

	network of [8]			degree network		
tier	$\Delta=3$	$\Delta=4$	$\Delta=8$	$\Delta=3$	$\Delta=4$	$\Delta=8$
A	55 440	112 860	458 100	55 440	112 860	112 860
B	44 356	90 291	366 482	44 357	90 291	90 292
C1	20 888	43 205	177 700	40 037	43 205	49 161
C2 [H]	4 905	10 470	46 182	9 080	9 799	19 350
C3 [H]	4 615	9 799	43 206	9 078	9 797	19 349
C4 [H]	4 330	8 695	39 079	8 021	8 024	15 846

Dominant term. At the tier-A crossover for $\Delta=3$ ($\ell=5$) the term $e_\ell U$ is $0.78846\,n$. The term $(e+2)gP/K$ is below $10^{-555}\cdot n$, $\log\tau$ below $10^{-4450}\cdot n$, and the grouping search and the branches below $10^{-12232}\cdot n$. The values of tier A are exactly $30\log_2 K_\ell$ ($30\cdot 1848$, $30\cdot 3762$, $30\cdot 15\,270$). In tier B the binding lower-order term is the copy-bit term $g^3=27K^{24}$ in U and gP, so $t^*\approx 24\log_2 K_\ell$. The degree network has the smaller U from $\Delta\ge 4$ ($8.885n$ against $9.30n$ at $\Delta=4$; at $\Delta=3$, $8.06n$ against $6.98n$). Because levels are discrete, it lowers tier A only at $\Delta=8$, by a factor 4.06.

3.5. **[O], Engine I (the interpolation algorithm): the Taylor term dominates.** We call Engine I the interpolation algorithm of `prop:interpolation-bound` of [O] (line numbers refer to `main_full_version.tex`). It counts in $(n+m)^{O_k(1)}2^{O_k(n(\log\log n)^2/\sqrt{\log n})}$ bit operations. Its proof sparsifies with $\sigma=1/s$, branches on variables occurring in more than $b_0=\lceil M/n\rceil t$ scopes, and runs the common counting routine (`prop:general-count`, l. 2013), whose cost is a sum of three terms (`eq:three-costs`, l. 2055). With $n=2^\lambda$ the rate is

$$\underbrace{\tfrac{1}{s}}_{\text{leaves}}+\underbrace{\tfrac{1}{t}}_{\text{branches}}+\underbrace{\tfrac{\log_2 C_q}{n}}_{\text{Taylor}}+\underbrace{2\tau\log_2(2D+1)}_{\text{measurements}}+\underbrace{\tfrac{2}{r}}_{\text{group}}+\frac{\text{polylog}}{n},\qquad C_q=\sum_{s'\le q}2^{s'}\binom{n}{s'},\quad q=\Bigl\lfloor\frac{d_H}{Q}\Bigr\rfloor,$$

with $d_H=kR(m_{\text{test}}+2n)=24Rn$ for $k=3$, $m_{\text{test}}\le 6n$. We bound $\log_2 C_q\le q+nH_2(q/n)+\log_2(q+1)$ for $q<2n/3$ and $C_q\le 3^n$ otherwise. Tier A takes the parameters of `lem:incidence-parameters` (l. 2747): $\lambda=\lceil\log_2 n\rceil$, $\nu=\lceil\log_2\lambda\rceil$, $r=\lfloor\sqrt{\lambda}/\nu^2\rfloor$, $Q=\lfloor\lambda/(128\nu)\rfloor$, $R=(\beta+3)\lceil\sqrt{\lambda}\rceil$ with β least with $b,w\le\lambda^\beta$, $\tau=1/(r\lambda)$, $D=r^{2Q}$, and the sparsification constants of `app:sparsify`. If any numerical check of the proof fails ($r\ge 2$, $Q>R$, $R\ge 32$, $6R\le Q$, $2(b+w)D^2\le M'$, $2Qr^2\le n'$, the tail bound), the algorithm searches exhaustively.

All checks first pass at $\lambda\approx 8.07\cdot 10^{10}$, where the rate is 1.599. The rate first drops below 1 at $\lambda^*=t^*\approx 3.0166\cdot 10^{13}$ ($\log_2\lambda^*=44.778$), with $\beta=6$, $r=2712$, $Q=5.24\cdot 10^9$, $R=4.94\cdot 10^7$, $q/n=0.2265$. The terms there are Taylor 0.9984, group $7.4\cdot 10^{-4}$, leaves $3.7\cdot 10^{-4}$, measurements $2.9\cdot 10^{-6}$ and branches $1.8\cdot 10^{-7}$. The polylogarithmic terms (Tanner search, $3n(2m)^{P_h}$ calls) are below $10^{-9\cdot 10^{12}}$. *The dominant term is the Taylor (Hasse-derivative) term* $\log_2 C_q$. The order $q=24Rn/Q$ is large because the window $R=(\beta+3)\lceil\sqrt{\lambda}\rceil$ comes from a Hoeffding bound [4] with a union over $b\le\lambda^\beta$ incidences, which is far wider than the binomial tail requires.

Tier B keeps the lemmas and only the numerical hypotheses of `lem:layout`. It optimises s, t, r, R, with Q at its largest allowed value and $\tau=r\,b\Pr[W\notin I]$ the least allowed, $W\sim\mathrm{Bin}(Q,(r-1)/(2r))$. We bound

TABLE 9. Engine I, tier A. For $\lambda \le 2^{36}$ a check fails ($s < 2$ up to 2^{18}; then $r < 2$ or $Q \le R$; then $6R > Q$) and the algorithm searches exhaustively.

$\log_2 \lambda$	38	42	44	46	50
β	7	6	6	6	6
r	363	1 188	2 166	3 964	13 421
q/n	2.23	0.554	0.290	0.152	0.0412
rate	1.593	1.548	1.16	0.7662	0.2892

the tail by the Chernoff–Kullback–Leibler bound, which is a rigorous upper bound on the exact binomial tail. The search runs in floating point (coordinate descent with restarts), and we re-evaluate each reported point in mpmath with the same cost function as tier A.

TABLE 10. Engine I, tier B: best rate found at $\lambda = 2^j$, and its parameters.

j	24	26	28	32	40
rate	1.5083	1.0699	0.7097	0.2807	0.0349
s	128	128	128	512	4 096
t	512	512	512	2 048	16 384
r	18	18	29	86	946
R	17 150	34 948	66 152	244 286	3 463 792

The tier-B crossover is $\lambda_B^* \approx 8.53 \cdot 10^7$ ($\log_2 \lambda^* = 26.35$), at which the mpmath rate is 0.99983 (Taylor 0.888, group 0.100, leaves 0.0078, branches 0.0020, measurements 0.0018). *The exact-tail window shrinks t^* by a factor of about $3.5 \cdot 10^5$.* In tier B the binding trade-off is between the group term $2/r$ and the Taylor term, through $Q_{\max} \propto \lambda / \log r$. Tier C ($m_{\text{test}} \approx 3.41n$, so $d_H \approx 16.2Rn$) gives $\lambda_C^* \approx 3.82 \cdot 10^7$ ($\log_2 \lambda^* = 25.19$, rate 0.99985). We did not use two further savings that the proofs do not give: halving the group term, and a lower precision than $\ell = n + 1$.

3.6. **[O], Engine II (tensor contraction): only a lower bound.** The sharper bounds of [O] come from `thm:tensor-contraction` (`quantitative-reduction.tex`, l. 26), all of whose rate constants are $O_C(\cdot)$. What is explicit is the parameter regime of `quantitative-contraction.tex`: $\varepsilon = 2^{-17}$, $u_d = \lfloor \varepsilon\lambda/2^d \rfloor$, and the requirement $\lambda^{4/5} \le u_d$ at every active call, where now λ is $\log_2$ of the number of wires. Active calls reach depth $d < (\log_2 C + \frac{1}{2}\log_2 \lambda)/3$. There the requirement reads $\varepsilon\lambda^{5/6}/C^{1/3} \ge \lambda^{4/5}$, that is, $\lambda \ge 2^{510}C^{10}$; at depth 0 alone it needs $\lambda \ge \varepsilon^{-5} = 2^{85}$. We read the parameter bounds as hypotheses that the proof needs. Then Engine II as written says nothing below $\log_2(\text{wires}) \approx 2^{85}$, which is far beyond Engine I's crossover, and it gives no finite upper bound on n^* at all. Engine I is therefore the operative crossover of [O].

3.7. **Which levers matter: the degree q first.** For [O] one lever dominates: the exact binomial window in place of Hoeffding (tier B), a factor $3.5 \cdot 10^5$ in t^*. The polynomial factors cannot matter. For [A] the levers are, in order:

(i) *the degree q*, existential and unbounded in Lean: about $\times 4$ in t^* per doubling;
(ii) *a fast base case* ($e_0 = 5/18$, [H]): e_ℓ times 0.556, about 1.7 levels, 119 103 → 31 358 (factor 3.8) in row 1;
(iii) *e_ℓ in place of e'_ℓ and a free level* (tier B, proven in other Lean routes): 552 040 → 245 009 (factor 2.25);
(iv) *re-tuning ζ_ℓ and g_ℓ/K_ℓ* (C1): 245 009 → 119 103 (factor 2.06); Lean's $\zeta_\ell = 2^{-(\ell+5)}$ has nearly the right shape, but $g_\ell = 2^{3\ell+x_0}K_\ell$ is larger than condition (ii) needs;
(v) minor: $x_0 = 10$ ([H], factor 1.18) and dropping the grouping search ([H], factor 1.14); $3/K_\ell$, C_0, C_1, C_2 are irrelevant.

From A to C2 the gain in row 1 is a factor 17.6. For #HC the same schedule levers apply.

4. Four facts about the sources

A first reading of the statements suggests four things that the sources, read in full, do not say. We record what they do say, because each changes a number above.

(1) *The decision machine's floor is* $2^{2^{1722}}$*, not* $2^{2^{518}}$. The docstring of `statements/ThreeSatRateStatement.lean` (line 64) says that the machine of the proof is exhaustive search for every $n < 2^{2^{518}}$. This is true but weak. With `sOf`, `sOK` and `COK` of `rates/PaddedDescent/Params.lean` (lines 56, 83, 87) the machine searches exhaustively for every $n < 2^{2^{1722}}$, and its pad exceeds 2^n until $n \geq 2^{2^{1747}}$, as `rates/PaddedDescent/Main.lean` itself states (lines 52–59).

(2) *The official theorem uses* $e'_\ell \approx 1.55\, e_\ell$*, not* e_ℓ. The official `MainResults.mainTheorem` (`Solution.lean`, line 44) is `CountedRoute.mainTheoremHolds`, whose solver `solverOfMachine` pads to `needOf`$= \lceil e' K^2 \lambda \rceil + C_0 K^2 \operatorname{size}(K(\lambda + B + 1)) + 2$ (`rates/CountedToPadded/Defs.lean`, line 148) with $e' = e'_\ell$ from `num`/`den` (`rates/GeneralPieces/Num/Between.lean`, lines 34–37). Since $e'_\ell \approx \rho^\ell$, it is about $1.55\, e_\ell$ from $\ell = 6$ on (Table 2). The interface `NumberBetween` (`statements/CliqueLevelsStatement.lean`) allows any number in $[e_\ell, 2e_\ell]$; tier B uses e_ℓ, its lower end. Separately, the same file says that in Turing-machine steps the exponent $e_\ell K_\ell^2 \lambda$ is claimed only up to a constant factor fixed with the machine (docstring, lines 25–33); in counted operations (`SolvesInFactor`) it is exact. We count the former through q.

(3) *OpenAI's occurrence bound is explicit.* `lem:sparsify` of [O] (l. 3543) states $B = O_k([\sigma^{-1} \log(2/\sigma)]^{k-1})$, but adds that B is "an explicitly computed integer", and `app:sparsify` ("The size of the tree", l. 3808) computes it: $\Gamma = 2(3^{k-1} - 1)$, M the least power of 2 with $M \geq \Gamma$ and $\Gamma \lceil \log_2(4CM) \rceil \leq \sigma M$, $C = 4 \sum_{j \leq k-2} (3M)^j$, $B = 2 \sum_{j<k} M^j$. We use these values; no hidden constant enters through B.

(4) *Anthropic's occurrence bound is explicit.* The n-route caps the number of occurrences per variable by `occBound` (`variables/RecursionToBound/Defs.lean`, line 115), $2 \sum_j \theta_j + 2 = 32\alpha^2 + 4\alpha + 6$ for $k = 3$, with α, β_c, θ_c defined in `sparsification/SparsificationLemma/Cal/Defs.lean`. It is an explicit function of ε and involves no existential constant.

5. Related work

We organise related work by idea: the ETH and its claimed refutations, the brute-force baselines, our Hamiltonian-cycle counter, and the two ingredients that enter only as levers.

The ETH and its claimed refutations. Impagliazzo and Paturi [5] stated the ETH. OpenAI [7] and Anthropic [1] claim deterministic subexponential algorithms for counting the models of a k-CNF, with Lean formalisations of the top-level statements; both are under review. Both state their bounds with $O(\cdot)$ in the exponent, and neither computes where its concrete bound crosses brute force. In contrast, we evaluate the proven bounds of both term by term, and report where they first beat brute force.

Brute-force baselines. For #3-SAT the baseline is the 2^n of exhaustive search. For #HC it is the $n^2 2^n$ of the dynamic programme of Bellman [2] and of Held and Karp [3]. In contrast to a comparison with the fastest exponential algorithms, we compare only with these simple bounds, as the question in the title asks.

Our Hamiltonian-cycle counter. In earlier work [8] we derived from [A] an exact deterministic algorithm that counts Hamiltonian cycles in time $2^{O(n/\sqrt{\log n})}$ on graphs with $m = O(n)$ edges, with a Lean development, and recorded its explicit bounds in research notes [9]. Closest to the present note is that work. In contrast, there we proved asymptotic bounds; here we evaluate their constants and compare them with Held–Karp.

Fast base cases and concentration bounds. Two neighbouring ingredients enter only as levers. A fast-matrix-multiplication base case, with $\omega = 2.371552$ [10], might lower the base clique exponent; we use it only in the heuristic tiers C2 and C3. The window of Engine I of [O] comes from a Hoeffding bound [4]; tier B replaces it by the Chernoff–Kullback–Leibler bound on the exact binomial tail. In contrast to a proof of a faster base case, we only report what such a base case would be worth, and we do not claim it.

6. Limitations

- *Everything is conditional.* The bounds evaluated here are those of the claimed refutations [A] and [O], and, for #HC, of our paper [8], which assumes three internal results of [A]. Expert review of the manuscripts is pending. If their proofs fail, the numbers lose their meaning.

- *Computations, not theorems.* We transcribed the formulas from Lean definitions and from the TEX sources (for [O], we read only the `.tex` files) and evaluated them with a Python script. Neither the transcription nor the script is formally verified, and no number in this note is machine-checked. The Lean development of [O] certifies no finite crossover at all.
- *Existential constants.* The degree q of [A]'s Turing-machine bound is unknown and unbounded in Lean. The values of rows 1–3 of Table 1 are those at $q = 1$, that is, lower bounds. The thresholds $n_0 \le n_1$ of the n-route can only increase its crossover. The constants C_0, C_1, C_2 and the polynomial degrees of [O] are shown to be irrelevant only within the stated break-even ranges.
- *Heuristic tiers.* The entries marked [H] (fast base case, $x_0 = 10$, no grouping search) rest on changes that nobody has proved. Tier C1 rests on the written proof of [A], and tier C of [O] on our reading of two proofs; neither is in Lean.
- *Engine II* of [O] yields only a lower bound. Its rate constants are $O_C(\cdot)$, and the threshold 2^{85} uses our reading of its parameter bounds as hypotheses of the proof.
- *Proven bounds, not running times.* A crossover of a proven upper bound says nothing about the actual running time, which may be far smaller. Tier B is only the best point found by a coordinate search, not a proven optimum. The #HC cost model uses the explicit bounds of our research notes, which accompany but are not identical with the Lean statements of [8].
- *Nothing practical follows.* The smallest crossover in this note, $t^* \approx 4\,330$ for #HC with all heuristic changes, is an input with more than 10^{1300} vertices.

7. Reproducibility

The computation is the Python script `research/scripts/crossover.py` in our public repository https://github.com/xennygrimmato/eth-hamiltonicity [9], at commit `34132bb`. Its notes are in `research/crossover.md`, and its output is `research/scripts/crossover.out`. The script and its output are in the repository; they are not attached to this submission. It is run as `python3 -I crossover.py` (isolated mode, so that no file of the working directory is imported) and writes `crossover.out` beside itself. It needs only the Python standard library and mpmath [6] at 60 significant digits (`mp.mp.dps = 60`). On 3 November 2026, with Python 3.13.17 and mpmath 1.3.0, a fresh run took 29 seconds of wall-clock time and reproduced the stored `crossover.out` byte for byte; a second fresh run on 4 November 2026 did the same. Every number in this note is either printed in that file or, where stated, a ratio or difference of printed values. The script also asserts the closed form of $\log_2 K_\ell$ and condition (ii) for the Lean schedule for $\ell < 40$.

Acknowledgements

This work was done with AI assistance (Claude Code, Anthropic): the transcription of the bounds, the script and the text were produced in sessions with Claude Code. The numbers have been checked against a fresh run of the script and the formulas against the sources as described, but the note awaits human expert review. Responsibility for any errors rests with the author.

References

[1] Claude (Anthropic), *A $2^{o(n)}$ time algorithm for 3-SAT*, preprint dated 26 October 2026. Paper and Lean formalisation in the repository https://github.com/anthropics/formal-math, folder `eth`.
[2] R. Bellman, *Dynamic programming treatment of the travelling salesman problem*, J. ACM **9** (1962), no. 1, 61–63.
[3] M. Held and R. M. Karp, *A dynamic programming approach to sequencing problems*, J. Soc. Indust. Appl. Math. **10** (1962), no. 1, 196–210.
[4] W. Hoeffding, *Probability inequalities for sums of bounded random variables*, J. Amer. Statist. Assoc. **58** (1963), no. 301, 13–30.
[5] R. Impagliazzo and R. Paturi, *On the complexity of k-SAT*, J. Comput. System Sci. **62** (2001), no. 2, 367–375.
[6] The mpmath development team, *mpmath: a Python library for arbitrary-precision floating-point arithmetic (version 1.3.0)*, 2023, https://mpmath.org/.
[7] OpenAI, *The Exponential Time Hypothesis is false*, preprint dated 17 October 2026, revised 25 October 2026. Paper and Lean formalisation in the repository https://github.com/openai/math.
[8] V. Tulsyan, *Counting Hamiltonian cycles in sparse graphs in subexponential time, assuming the new #SAT algorithms*, Hexagon preprint hexagon:2610.00672v1, https://hexagonmath.org/abs/2610.00672v1, October 2026; source and Lean development at https://github.com/xennygrimmato/eth-hamiltonicity.

[9] V. Tulsyan, *eth-hamiltonicity: Lean 4 formalisation and research notes*, public repository https://github.com/xennygrimmato/eth-hamiltonicity, 2026; scripts, research notes and Lean sources as of commit `34132bb`.

[10] V. Vassilevska Williams, Y. Xu, Z. Xu and R. Zhou, *New bounds for matrix multiplication: from alpha to omega*, Proc. 2024 Annual ACM-SIAM Symposium on Discrete Algorithms (SODA), SIAM, 2024, pp. 3792–3835; arXiv:2307.07970.